

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

**Escuela Profesional de Ingeniería de Sistemas y
Computación**

Asignatura: Algoritmos y Estructuras de Datos

Mg. Jaime Ortiz Fernández

e-mail: d.jortiz@ms.upla.edu.pe

TEMA: Estructuras Dinámicas.
Pilas

Diapositivas: Mg. Jaime Ortiz Fernández

Pilas o Stacks

- Tipos de Datos Abstractos (TDAs)
- Pilas (Stacks)
- Aplicación al análisis lexicográfico
- Implementación Java de una pila
- Interfaces y excepciones



Tipos de Datos Abstractos (TDAs)

Un **Tipo de Dato Abstracto (TDA)** es una abstracción de una estructura de datos. (No hay código relacionado)

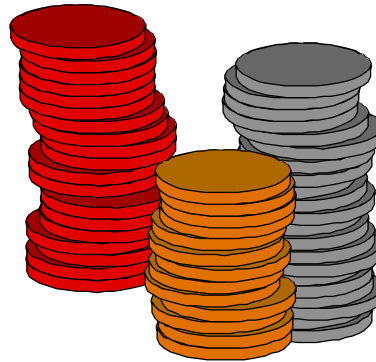
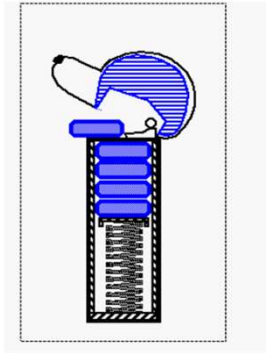
El **TDA** especifica:

- **qué se puede almacenar** en el TDA
- **qué operaciones** se pueden realizar sobre o por el TDA

Por ejemplo, si se modela una bolsa de caramelos como un TDA, se puede especificar que:

- este TDA almacena caramelos
- este TDA permite poner un caramelo
- Este TDA permite extraer un caramelo.
- Este TDA permite sacar todos los caramelos o vaciar la bolsa

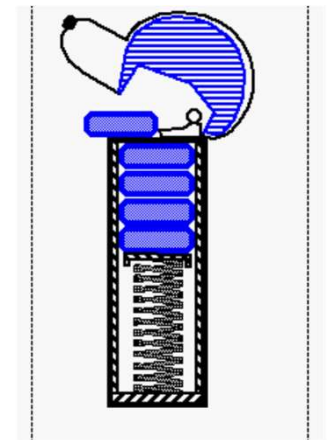
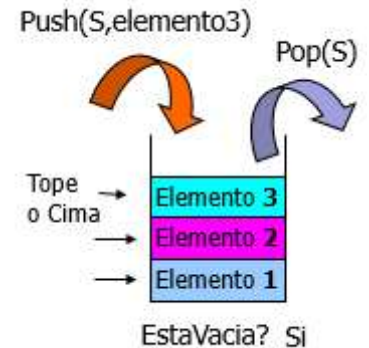
Ejemplo



- Hay una gran cantidad de TDAs formalizados y estándar.
- En lo sucesivo se mostrarán varios TDAs estándar diferentes (*pilas*, colas, árboles...)

Pilas (Stacks)

- Una pila es un contenedor de objetos, donde los objetos se insertan y extraen de acuerdo al principio de *último en entrar, primero en salir* (**last-in-first-out, LIFO**).
- Una pila (stack en inglés) es una estructura de datos lineal que solo tiene un único punto de acceso fijo por el cual se añaden, se eliminan o se consultan sus elementos. El modo de acceso a los elementos es de tipo **LIFO** (del inglés **Last In First Out**, último en entrar, primero en salir).
- Los objetos se pueden insertar en cualquier momento, pero sólo el último objeto (el insertado más reciente), puede ser extraído.
- La inserción de un elemento se conoce como “pushing o push” en la pila. “Popping o pop” de la pila es sinónimo de extraer un elemento.

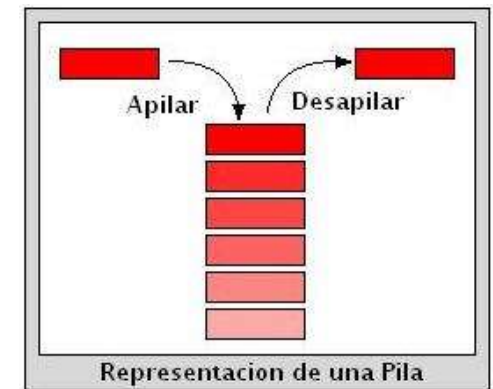


Características

- La única forma de acceder a los elementos es desde el tope o cima de la pila.
- Su administración es muy sencilla ya que tiene pocas operaciones.
- Si la pila esta vacía no tiene sentido referirse a un tope ni a un fondo.
- En caso de querer acceder a un elemento que no se encuentre en el tope de la pila se debe realizar un volcado de la pila a una pila auxiliar, una vez realizada la operación con el elemento se vuelve a volcar los elementos de la pila auxiliar a la original.

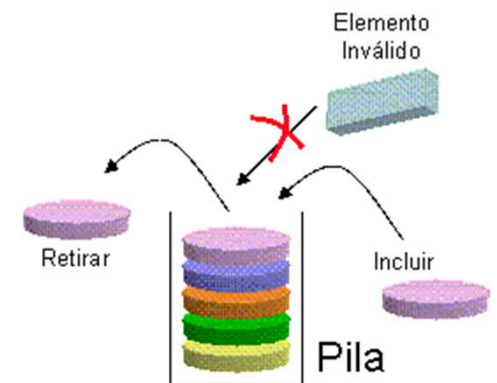
El Tipo de Dato Abstracto Pila

- Una pila es un **tipo de dato abstracto** (TDA) que soporta dos métodos principales:
 - **push(o)**: Inserta un objeto sobre el ultimo objeto apilado o en la cima de la pila.
 - **pop()**: Extrae el objeto de la cima de la pila y lo devuelve; **si la pila está vacía, ocurre un error.**
- Los siguientes métodos para la gestión de la pila deben ser definidos:
 - **size()**: Devuelve el número de objetos en la pila.
 - **isEmpty()**: Devuelve un boolean indicando si la pila está vacía.
 - **top()**: Devuelve el objeto de la cima de la pila sin extraerlo; **si la pila está vacía, ocurre un error.**



Operaciones

- apilar (objeto o valor): también conocido como **push** agrega el valor al tope de la pila.
- retirar (): también conocido como **pop** retira el último elemento apilado.
- cima (): devuelve el valor del elemento que está en la cima de la pila.
- esVacia (): retorna true si la pila no ha sido inicializada o no tiene datos.
- buscar (valor): retorna la true si el elemento a buscar existe en la pila.
- eliminar(): elimina la pila
- listar (): imprime en pantalla los elementos de la pila.
- tamaño(): devuelve el número de objetos que tiene la pila



PILA: OPERACIONES

Pop(Pila) retorna -> *elemento* **DESAPILAR**

Efecto: Recibe la pila, remueve el elemento tope y lo **retorna**

Excepcion: Si la pila esta vacía, produce error

Push(pila, elemento) **APILAR**

Efecto: Toma la pila y aumenta su tamaño, poniendo el elemento en la cima de la pila

InicializarPila(Pila)

Efecto: recibe una pila y la inicializa para su trabajo normal

EliminarPila(Pila)

Efecto: recibe una pila y la libera completamente

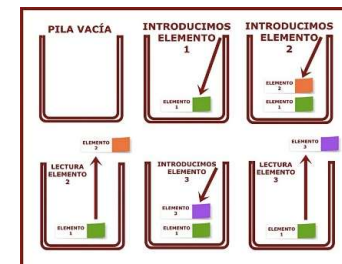
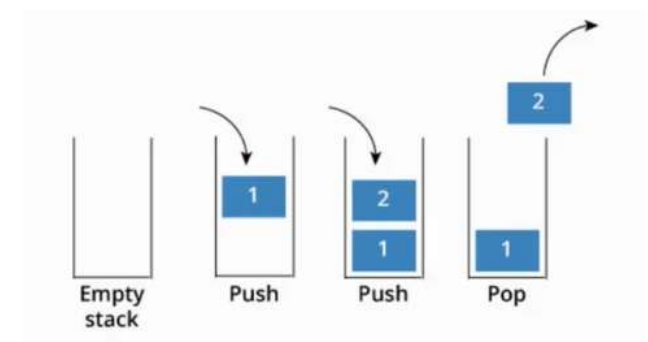
EstaVacía(Pila) retorna -> Boolean

Efecto: Devuelve true si esta vacía y false en caso contrario

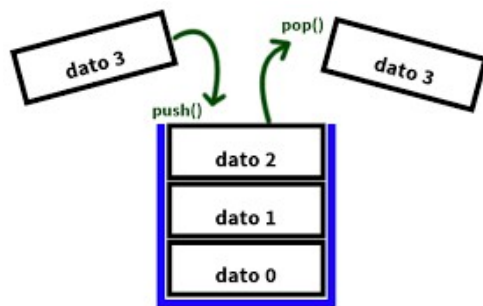
TopePila(Pila) retorna -> *elemento*

Efecto: Devuelve el elemento cima de la pila

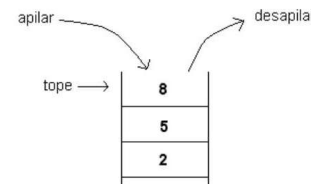
Excepcion: Si la pila esta vacía produce error



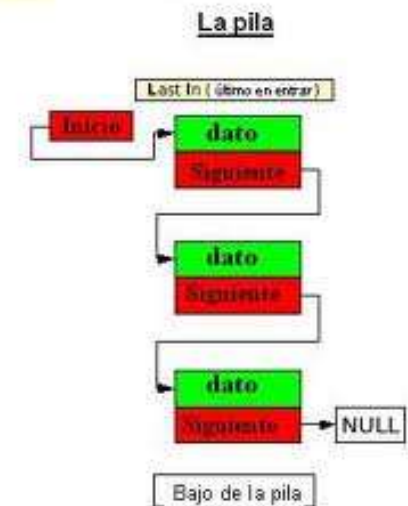
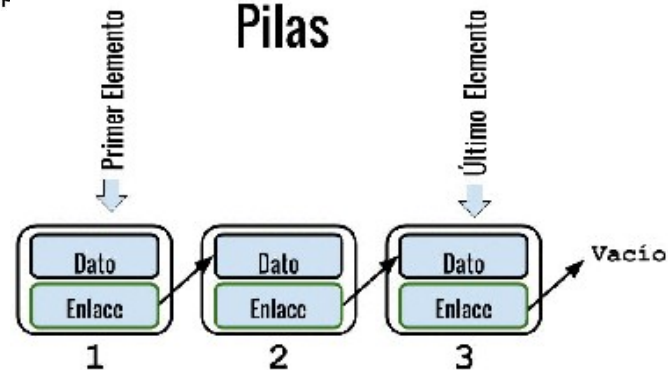
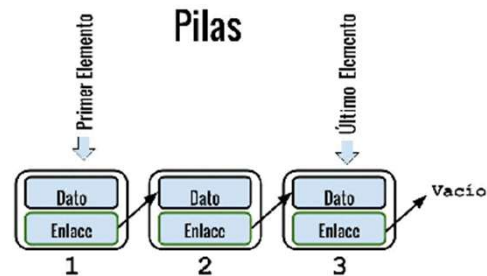
El tratar de remover elementos o acceder a elementos de una pila vacía se llama SUBDESBORDAMIENTO de la pila



- PUSH: apilar, meter
- POP: desapilar, sacar
- TOP: cima, tope



Pilas



Una Interface Pila en Java

Mientras, la estructura de datos pila viene en una clase “predefinida” en el paquete `java.util`, definimos nuestra propia interface Stack:

```
public interface Stack {  
  
    // metodos de acceso  
    public int size();  
    public boolean isEmpty();  
    public Object top()    throws StackEmptyException;  
  
    // metodos de actualizacion  
    public void push (Object element);  
    public Object pop()    throws StackEmptyException;  
}
```

Aplicaciones de las pilas

Gran uso en compiladores y SO's.

Entornos donde haya que recuperar el último valor que se almacenó (*backtracking*)

Algunas aplicaciones:

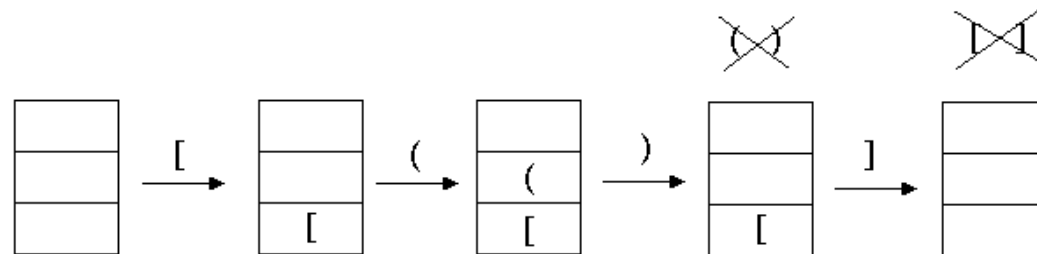
- Equilibrado de símbolos
- Llamadas a subprogramas
- Eliminación de recursividad
- Tratamiento de expresiones aritméticas
 - ✓ Evaluación de expresiones postfijas
 - ✓ Conversión infija a postfija
- Borrado de caracteres en un editor de textos

Equilibrado de símbolos

Se van leyendo los caracteres. Cuando se encuentra un elemento clave (paréntesis, corchete...) se trata según su tipo:

- Si es de apertura: se mete en la pila.
- Si es de cierre:
 - Si la pila está vacía $\rightarrow$ error.
 - Si la pila no está vacía:
 - Si la cima es el correspondiente símbolo de apertura se extrae.
 - Si no lo es $\rightarrow$ error.

Si al final la pila no está vacía $\rightarrow$ error



Llamadas a subprogramas

Al llamar a un subprograma se necesita guardar:

- Estado de las variables locales del programa que llama
- Dirección del programa en la que se hizo la llamada

} *Registro de
activación*

Al hacer la llamada esta información se mete en la cima de una pila.

Al terminar el subprograma, se saca la cima y se recupera el estado del momento de la llamada que vuelve al punto de ejecución donde se hizo la llamada.

El subprograma puede llamar a otros subprogramas y así sucesivamente.

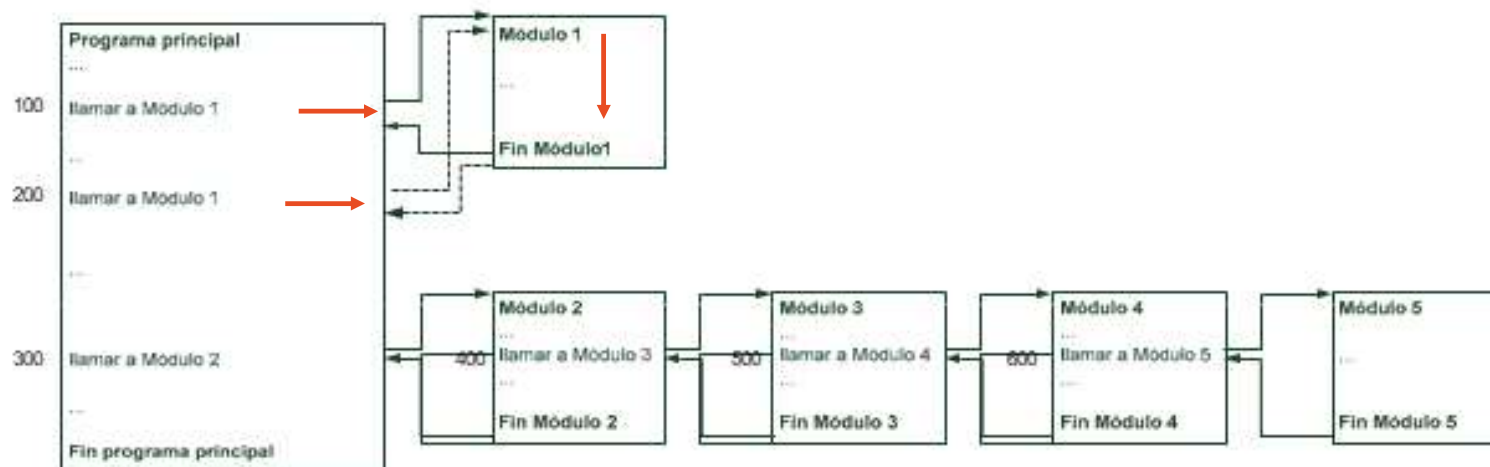
Permite implementar la recursión.

Peligro: rebasamiento de la pila

Llamadas a módulos

Un programa modular contará con un programa principal y uno o varios módulos.

El programa principal llama o invoca a los módulos, cediendo a éstos el control del flujo del programa y al terminar devuelve el control al programa o módulo que hizo la llamada.



Eliminación de recursividad

La recursión consume muchos recursos (memoria).

Recursión de cola: la llamada recursiva está en la última línea. Para eliminarla:

- Se necesita: argumentos del algoritmo pasados por valor o referencia si son los mismos argumentos los que se pasan a la llamada recursiva.
- Se asignan a los argumentos los valores que se van a pasar en la llamada recursiva.
- Salto (*goto*) al principio de la rutina.

Para transformar algoritmo recursivo en iterativo:

- Se guarda en pilas el estado del problema en el momento de la llamada recursiva.
- Se vuelve al principio de la rutina mediante una estructura iterativa.
- La vuelta atrás de la recursión se consigue sacando los valores de las pilas

Tratamiento de expresiones aritméticas

Notación infija: $a + b$

Notación prefija: $+ a b$

Notación postfija: $a b +$

Problema: distinción de prioridades en notación infija. Ej: evaluar $a + b * c$

Soluciones:

- Empleo de paréntesis.
- Conversión a notación prefija o postfija.

Ventajas de la notación postfija:

- No hace falta conocer reglas de prioridad.
- No hace falta emplear paréntesis.

Borrado de caracteres en un editor de texto

- Se van leyendo los caracteres de uno en uno.
- Si el carácter no es de borrado ni de eliminación de línea, se mete en la pila.
- Si el carácter es de borrado, se hace un *pop()*, para sacar el elemento cima de la pila.
- Si el carácter es de eliminación de línea se vacía toda la pila.

Ejemplo: si # es el carácter de borrado,

in p # f o r o n # # m á t i z # c a —————→ *informática*

Una Pila basada en Array

Crea una pila usando un array especificando un tamaño máximo N para la pila, p.e., $N = 1024$.

La pila consiste de un array S de N -elementos y una variable entera t que es el índice al elemento de la cima en el array S .



NOTA: Los índices del Array empiezan en 0, por lo que se inicializa t a -1

Pseudocódigo en la derecha.

Algoritmo size():

return $t + 1$

Algoritmo isEmpty():

return $(t < 0)$

Algoritmo top():

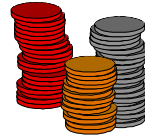
if isEmpty() then

throw a StackEmptyException

return $S[t]$...

Pseudocódigo

```
Algoritmo push(o): // apilar
    if size() = N then
        throw a StackFullException
    t ← t + 1
    S[t] ← o
```



```
Algoritmo pop(): // desapilar
    if isEmpty() then
        throw a StackEmptyException
    e ← S[t]
    S[t] ← null
    t ← t-1
    return e
```



Una Pila basada en Array

La implementación basada en array es simple y eficiente.

Hay un límite superior predefinido, N , del tamaño de la pila, que puede ser muy pequeño para una aplicación dada, o causar un desperdicio de memoria.

Excepciones:

StackEmptyException Excepción de pila vacía, no se pueden extraer elementos

StackFullException Excepción de pila llena, no se pueden agregar elementos.

StackEmptyException Excepción de pila vacía, no se pueden leer elementos

Pila basada en Array en Java

```
public class ArrayStack implements Stack {  
    // Implementacion de la interface Stack usando un array.  
    public static final int CAPACITY = 1024; // capacidad de la pila por defecto  
  
    private int capacity; // maxima capacidad de la pila  
  
    private Object S[ ]; // S mantiene los elementos de la pila  
  
    private int top = -1; // elemento cima de la pila  
  
    public ArrayStack( ) { // Inicializa la pila  
        this(CAPACITY); // con la capacidad por defecto  
    }  
  
    public ArrayStack(int cap) { // Inicializa la pila con la capacidad dada  
        capacity = cap;  
        S = new Object[capacity];  
    }  
}
```

Pila basada en Array en Java (1)

```
public int size( ) { //Devuelve el tamaño en curso de la pila  
    return (top + 1);}
```

```
public boolean isEmpty( ) { // Devuelve true si la pila esta vacia  
    return (top < 0);}
```

```
public void push(Object obj) throws StackFullException{  
    // Push un nuevo elemento en la pila  
    if (size() == capacity)  
        throw new StackFullException("pila llena.");  
    S[++top] = obj;}
```


Pila basada en Array en Java (2)

```
public Object top() // Devuelve el elemento de la cima
    throws StackEmptyException {
    if (isEmpty()) throw new StackEmptyException("Pila vacia.");
    return S[top];}
```

```
public Object pop() // Pop extrae el elemento de la cima
    throws StackEmptyException {
    Object elem;
    if (isEmpty()) throw new StackEmptyException("Pila vacia.");
    elem = S[top];
    S[top--] = null; // Dereferencia S[top] y decrementa top
    return elem;
}
```

Pilas en la Java Virtual Machine

Cada proceso ejecutado en un programa Java tiene su propia *Java Method Stack*.

Cada vez que se invoca un método, se inserta en la pila (stack).

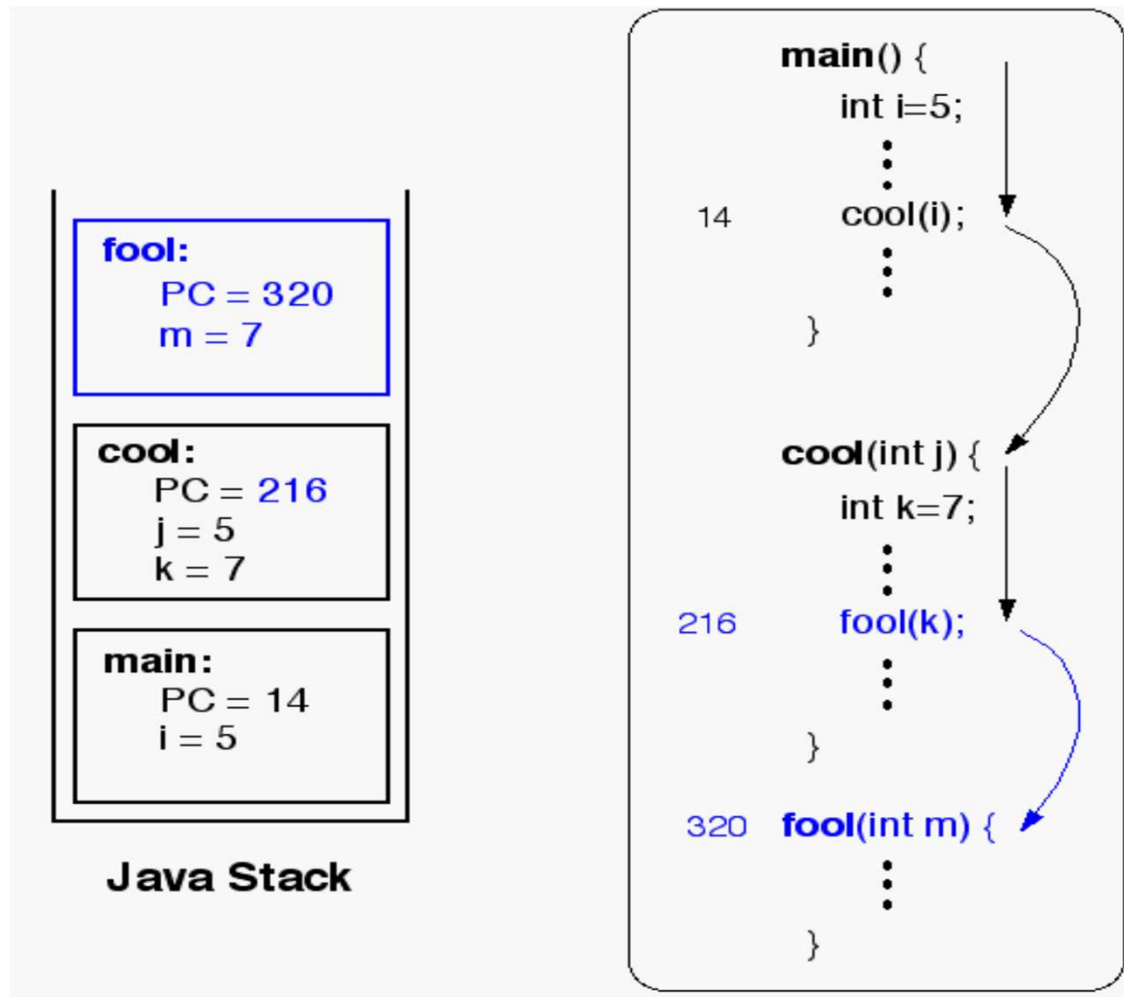
El uso de una pila para esta operación permite que Java realice varias cosas útiles:

- Realizar llamadas a métodos recursivos
- Imprimir trazas de la pila para localizar un error

Java también incluye una pila de operandos que se usa para evaluar instrucciones aritméticas, p.e.

```
Integer add(a, b):  
  OperandStack Op  
  Op.push(a)  
  Op.push(b)  
  temp1 ← Op.pop()  
  temp2 ← Op.pop()  
  Op.push(temp1 + temp2)  
  return Op.pop()
```

Pila de métodos en Java



UN APLICACIÓN PRACTICA – Análisis Lexicográfico

El compilador siempre sabe cuando se ha escrito un paréntesis, o una llave demás

- *¿Como lo hace?*

Con el uso de pilas, expresiones escritas:

$(a+b))$

Mal

$((a+b) * c / 4 * g - h)$

OK

Se puede reconocer los paréntesis que no coinciden

¿Como lograr esta aplicación de la pila?

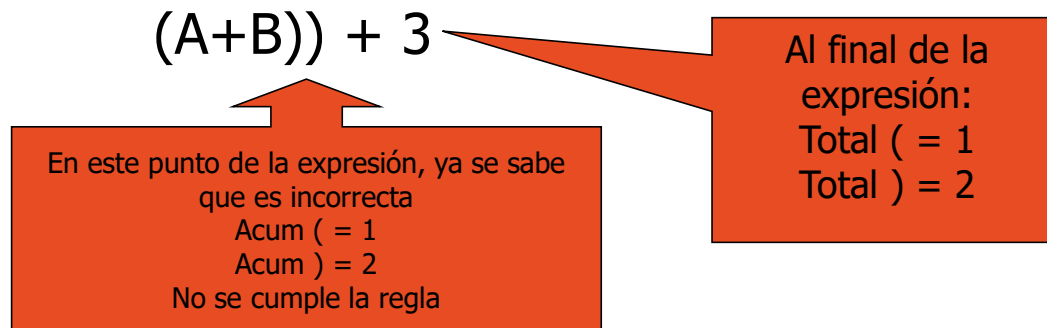
ANALISIS DEL PROBLEMA

$$7 - ((X * ((X + Y) / (J - 3)) + Y) / (4 - 2.5))$$

Cuando los paréntesis coinciden:

- Al final de la expresión
Total paréntesis izq = Total paréntesis der y
- En todo momento, en cualquier punto de la expresión
Cada paréntesis der. esta precedido de uno izq
Acum. paréntesis der. siempre es $\leq$ que Acum. paréntesis izq

Por ejemplo:



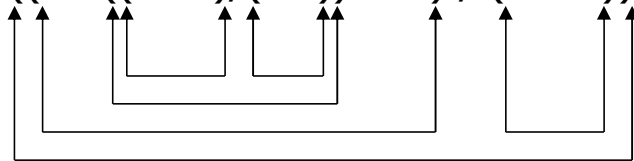
PRIMER ENFOQUE

- Podemos llevar un *conteo de paréntesis*
- Este debería llevar totales de) y de (
Acum. paréntesis Izq - Acum. paréntesis Der
- Este valor siempre deberá ser positivo
- Si en algún momento, al revisar la expresión, el conteo de paréntesis es negativo:
 - **BINGO**, hay un error

```
valida = true;
while(no hayamos revisado toda la
expresion)
{
    if (elemento_actual == '(')
        Total_der++;
    else if(elemento_actual == ')')
        Total_izq++;
    if(Total_der > Total_izq){
        valida = false;
        break;
    }
}
if (Total_der != Total_izq)
    valida = false;
```

UN ENFOQUE MAS NATURAL

- Otra forma, es la “forma natural”
- Cuando revisamos una expresión de este tipo:
 - Revisamos los paréntesis de la derecha, y buscamos si tienen “match” en la izquierda

$$7 - ((X * ((X + Y) / (J - 3)) + Y) / (4 - 2.5))$$


La Pila se utiliza justamente para “recordar” de la forma abajo indicada

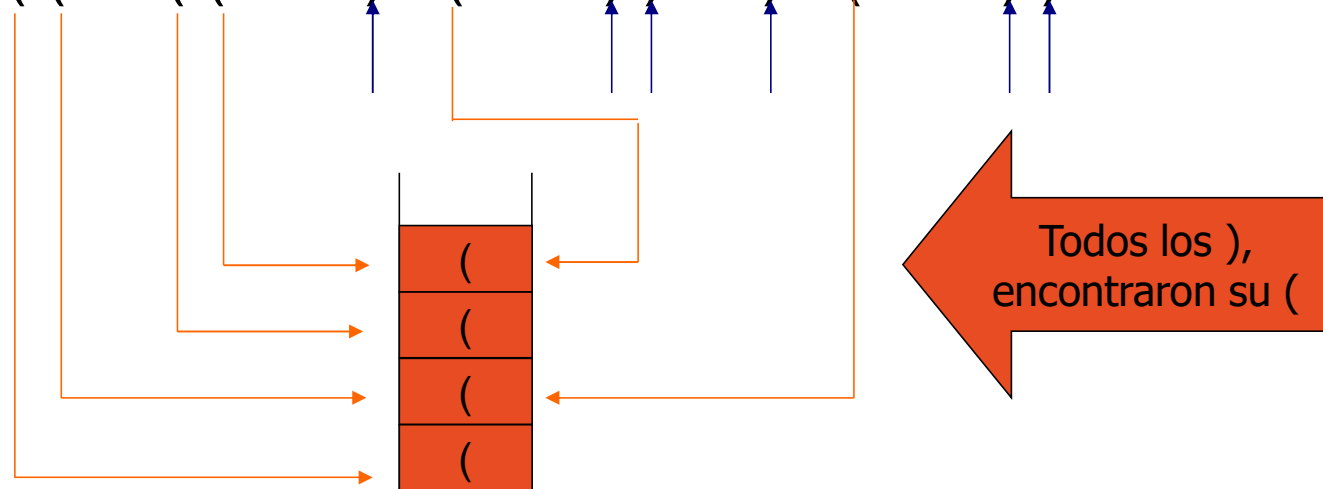
- Para seguir este enfoque podríamos:
 - “Recordar” los parentesis de la izquierda (a medida que aparecen:
 - El primero en aparecer, será el ultimo en ser “recordado”
 - El ultimo en aparecer, será el primero en ser “recordado”
- Así, cuando aparece un)
 - El primer (recordado, debe ser su “match”
 - En ese momento, este primero recordado, ya puede ser “olvidado”
 - Al llegar al final de la expresión, ningún (debería ser “recordado”

Pop el
paréntesis)
encontrado

APLICANDO PILAS

- Veamos, revisemos justo la expresión anterior

7 - ((X * ((X+Y) / (J- 3)) + Y) / (4- 2))



Y AHORA, EL ALGORITMO

```
Pila S; /*Se declara una pila s, para almacenar los ( */
Pila_Inicializar(&S);
while(no hayamos leído toda la cadena)
{
    //tomar el siguiente simbolo de la expresion
    if(simbolo = '(') /*Almacenarlo*/
        Pila_Push(&S,simbolo);
    if(simbolo = ')'){ /*Buscar match*/
        if(Pila_EstaVacía(S)) { /*No hubo match!!*/
            return(FALSE)
        }
        Pila_Pop(&s);
    }
}
if(Pila_EstaVacía(s)) /*Algún simbolo se quedó dentro,
    porque no hubo match*/
    return TRUE;
Else
    return FALSE;
```

La clase Stack

- La clase **Stack** es una clase de las llamadas de tipo **LIFO** (Last In - First Out, o último en entrar - primero en salir). Esta clase hereda de la clase **Vector** y con 5 operaciones permite tratar un vector a modo de pila o stack.
- Las operaciones básicas son **push** (que introduce un elemento en la pila), **pop** (que saca un elemento de la pila), **peek** (consulta el primer elemento de la cima de la pila), **empty** (que comprueba si la pila está vacía) y **search** (que busca un determinado elemento dentro de la pila y devuelve su posición dentro de ella).
- Esta clase es muy sencilla y al crear un objeto de tipo **Stack** con el constructor básico evidentemente no contendrá ningún elemento.
- Un conjunto mucho más completo y consistente para operaciones de [stack](#) LIFO es proporcionado en la interface **Deque** y sus implementaciones, pero de momento vamos a limitarnos al estudio de la clase **Stack**.

Métodos de la clase Stack

`empty()`: Regresa valor booleano `true` si la pila se encuentra vacía.

`push(item)` : Introduce el objeto `item` dentro de la pila.

`pop()`: Remueve el objeto de el tope de la pila

`peek()`: Muestra el objeto del tope de la pila sin extraerlo.

`search(item)`: Regresa un entero con la posición de el objeto `item` buscado o regresa un `-1` si no se encuentra el objeto.

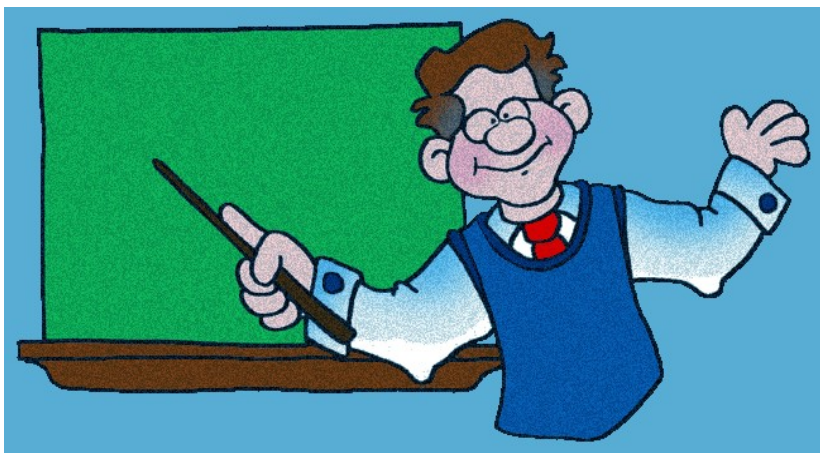
- Para crear la pila se usa:

```
Stack <Objeto> p = new Stack<Objeto>();
```

```
Stack <PilaCar> p = new Stack<PilaCar>();
```

```
Stack <Character> pcar = new Stack<Character>();
```

```
Stack <Integer> pint = new Stack<Integer>();
```



Gracias por su Atención



Preguntas?